

Learn Low-Level Design in 25 Days: Day 15— Designing a Notification Service (Email, SMS, Push)

How modern systems notify millions of users instantly— and how you can design one too.

TechPreneur

Published October 27, 2025

Free: No

Read on Freedium · original

Contents

1. The Story: When "Send Notification" Becomes a Nightmare 0
2. The Core Concept: The Notification Abstraction 0
3. The Factory: Decoupling the "What" from the "How" 0
4. Real-World Twist: Async Processing and Queues 0
5. Class Relationship Diagram (ASCII Visual) 0
6. Mini Case Study: How Uber Does It 0
7. Try It Yourself 0
8. Summary: What You Learned Today 0
Next in the Series 0

1. The Story: When "Send Notification" Becomes a Nightmare

You've built a great product. A user signs up — you send them a welcome email. They reset their password — you send an SMS. A new feature launches — you send a push notification.

Seems simple, right? Until… Marketing wants promotional emails. Ops wants system alerts via SMS. The mobile team adds Firebase push notifications. And now, every service in your codebase is directly calling some `EmailSender`, `SmsSender`, and `PushNotifier`.

One day, you find yourself buried under 3,000 lines of tangled code just to send one "Hello, World" message in three formats.

Welcome to the Notification Chaos.



In today's **Day 15** of Learn Low-Level Design in 25 Days, we're going to bring order to that chaos. You'll learn how to design a **scalable, extensible Notification Service** — one that can handle **Email, SMS, and Push** without breaking a sweat (or your architecture).

By the end of this article, you'll understand how platforms like **Slack, Amazon, or Uber** design their notification pipelines — and how to model yours using **interfaces, factory patterns, and queue-based dispatching**.

2. The Core Concept: The Notification Abstraction

Let's start simple. Every notification — whether it's an **email**, **SMS**, or **push** — has three things in common:

- A **receiver** (who gets it)
- A **message** (what's sent)
- A **channel** (how it's sent)

So instead of hardcoding `sendEmail()` or `sendSMS()`, let's define a common interface:

```
public interface NotificationSender {
    void sendNotification(Notification notification);
}
```

Here's a simple data class for our message payload:

```
public class Notification {
    private String recipient;
    private String message;
    // could include subject, metadata, or priority
}
```

Now, we create separate implementations for each channel:

```
public class EmailNotificationSender implements NotificationSender {
    public void sendNotification(Notification notification) {
        System.out.println("Sending EMAIL to " + notification.getRecipient());
    }
}

public class SMSNotificationSender implements NotificationSender {
    public void sendNotification(Notification notification) {
        System.out.println("Sending SMS to " + notification.getRecipient());
    }
}

public class PushNotificationSender implements NotificationSender {
    public void sendNotification(Notification notification) {
        System.out.println("Sending PUSH notification to " + notification.getRecipient());
    }
}
```

This pattern ensures **extensibility** — you can add new channels without touching your existing business logic.

3. The Factory: Decoupling the "What" from the "How"

If every service decides which sender to use, you'll end up with dozens of `if-else` or `switch` statements scattered across your code.

That's brittle, hard to test, and violates the **Open-Closed Principle**.

Enter the **Factory Pattern** — a clean way to delegate creation logic:

```
public class NotificationFactory {
    public static NotificationSender getSender(NotificationType type) {
        switch (type) {
            case EMAIL: return new EmailNotificationSender();
            case SMS: return new SMSNotificationSender();
            case PUSH: return new PushNotificationSender();
            default: throw new IllegalArgumentException("Invalid type");
        }
    }
}
```

Now your client code looks elegant:

```
Notification notification = new Notification("user@example.com", "Welcome to our app!");
NotificationSender sender = NotificationFactory.getSender(NotificationType.EMAIL);
sender.sendNotification(notification);
```

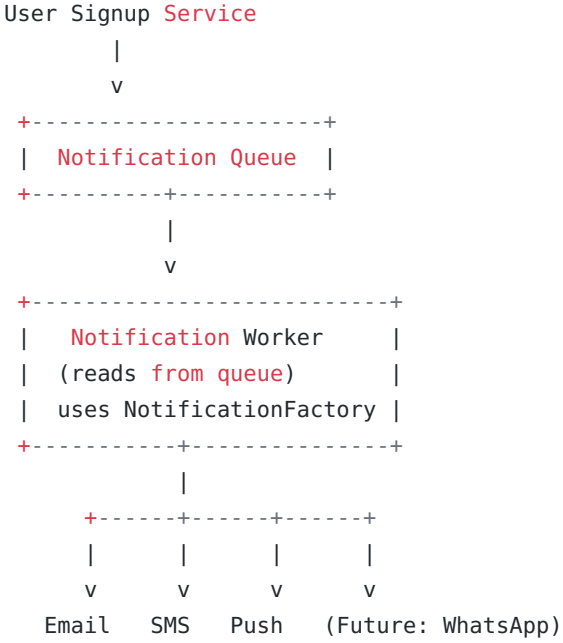
Need to add WhatsApp notifications next week? Just create `WhatsAppNotificationSender` and plug it into the factory. No core changes — no chaos.

4. Real-World Twist: Async Processing and Queues

In production, notifications aren't sent synchronously. You don't want your signup API waiting for an email server response.

Instead, real-world systems use **asynchronous processing** with a **message queue** (Kafka, RabbitMQ, AWS SQS, etc.).

Here's how it typically works:

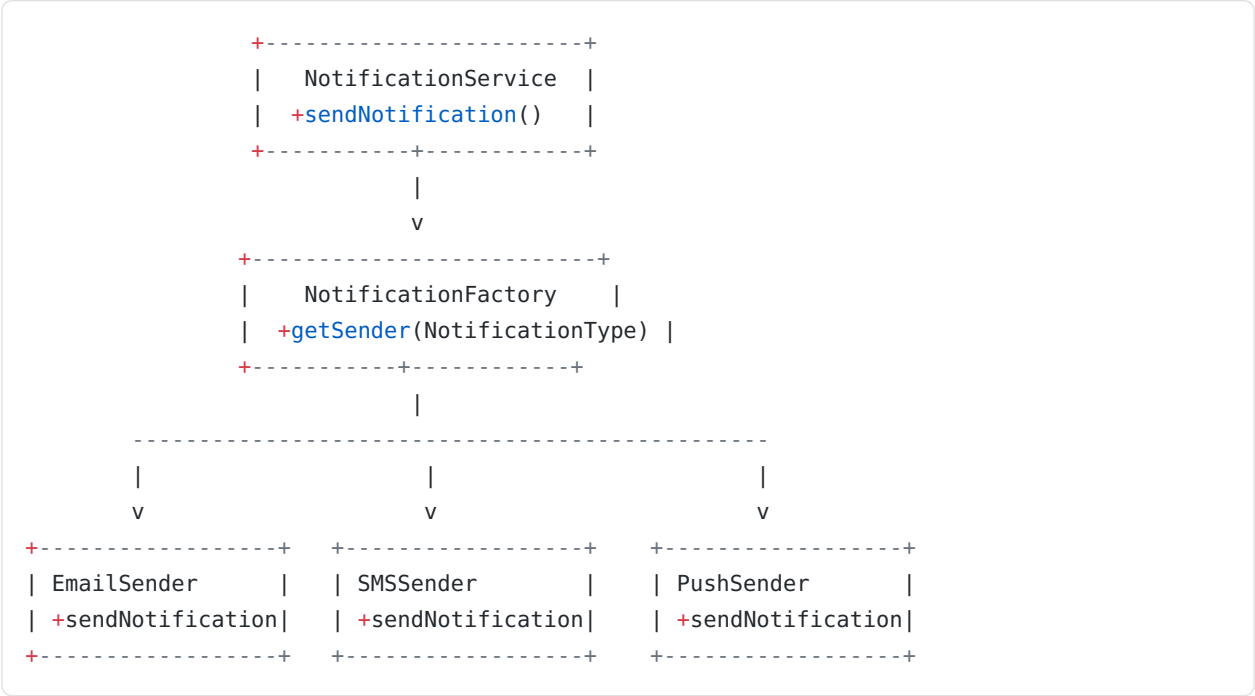


This architecture allows:

- **Loose coupling** (business logic doesn't care how notifications are sent)
- **Scalability** (workers can scale horizontally)
- **Reliability** (failed messages can be retried)

It's exactly how large-scale systems like **Uber**, **Slack**, and **Amazon SES** operate under the hood.

5. Class Relationship Diagram (ASCII Visual)



This is the essence of **polymorphism** in LLD — the client (`NotificationService`) doesn't care which sender it gets. It just calls `sendNotification()`, and the right implementation takes over.

6. Mini Case Study: How Uber Does It

When you book a ride, Uber might send:

- A **push** to the driver ("Your rider is waiting")
- An **SMS** to the passenger ("Your driver is arriving")
- An **email** receipt after the ride

All triggered by one event: `RideCompleted`.

Uber's backend uses an **event-driven architecture**: one event triggers multiple asynchronous notifications, each handled by specialized workers.

Your design here mirrors that — just on a smaller scale.

7. Try It Yourself

Task 1: Add a new `WhatsAppNotificationSender` and plug it into the factory.

Task 2: Implement user preferences. If a user opts out of SMS, skip SMS notifications.

Task 3 (Advanced): Simulate a queue-based asynchronous system — push notifications into a list, then process them with a separate thread or worker.

8. Summary: What You Learned Today

- **Notification Service Design** helps handle multi-channel communication cleanly.
- Use **interfaces** and **factory patterns** to abstract away delivery logic.
- Make notifications **asynchronous** using queues for scalability.
- This pattern is used by **Slack, Uber, Amazon, and almost every enterprise system**.
- You've learned to design for **extensibility** — a key principle of clean architecture.

When to Use:

- Whenever your app sends different types of messages or alerts.
- When you need scalable, pluggable channels (Email, SMS, Push, etc.).

When to Avoid:

- For small, single-channel tools or scripts (too much overhead).
- When simplicity is more important than flexibility.

Next in the Series

Tomorrow, we'll dive into something every backend engineer needs to master — **Database Layer Design: Repository & DAO Patterns.** You'll learn how to separate your business logic from persistence cleanly and elegantly.

Follow the full **"Learn Low-Level Design in 25 Days"** series here: https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8 (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)